



```
In [1]: #Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

```
In [3]: df = pd.read_csv(r"C:\Users\laxmi\Downloads\archive (4)\emails.csv")

print("Dataset loaded successfully!")
print("Shape:", df.shape)
print(df.head())
```

Dataset loaded successfully!

Shape: (5172, 3002)

	Email No.	the	to	ect	and	for	of	a	you	hou	...	connevey	jay	\
0	Email 1	0	0	1	0	0	0	2	0	0	...	0	0	
1	Email 2	8	13	24	6	6	2	102	1	27	...	0	0	
2	Email 3	0	0	1	0	0	0	8	0	0	...	0	0	
3	Email 4	0	5	22	0	5	1	51	2	10	...	0	0	
4	Email 5	7	6	17	1	5	2	57	0	9	...	0	0	

	valued	lay	infrastructure	military	allowing	ff	dry	Prediction
0	0	0		0	0	0	0	0
1	0	0		0	0	0	1	0
2	0	0		0	0	0	0	0
3	0	0		0	0	0	0	0
4	0	0		0	0	0	1	0

[5 rows x 3002 columns]

```
In [5]: #Separate features and target

X = df.iloc[:, 1:-1]
y = df.iloc[:, -1]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

X shape: (5172, 3000)

y shape: (5172,)

```
In [6]: #Check the classes
print("Class distribution:")
print(y.value_counts())

print("\n0 = Not Spam")
print("1 = Spam")
```

```
Class distribution:
Prediction
0    3672
1    1500
Name: count, dtype: int64
```

0 = Not Spam

1 = Spam

```
In [7]: #plit the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

Training data: (4137, 3000)

Testing data: (1035, 3000)

```
In [8]: #Standardize the features
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Feature scaling completed!")
```

Feature scaling completed!

```
In [9]: #K-Nearest Neighbors
knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train_scaled, y_train)

y_pred_knn = knn.predict(X_test_scaled)

print("KNN classification completed!")
```

KNN classification completed!

```
In [10]: #KNN performance
accuracy_knn = accuracy_score(y_test, y_pred_knn)
precision_knn = precision_score(y_test, y_pred_knn)
```

```

recall_knn = recall_score(y_test, y_pred_knn)
f1_knn = f1_score(y_test, y_pred_knn)

print("KNN Performance")
print("-----")
print("Accuracy :", round(accuracy_knn, 4))
print("Precision:", round(precision_knn, 4))
print("Recall   :", round(recall_knn, 4))
print("F1 Score :", round(f1_knn, 4))

```

```

KNN Performance
-----
Accuracy : 0.8338
Precision: 0.6435
Recall   : 0.9567
F1 Score : 0.7694

```

```

In [11]: #KNN confusion matrix
cm_knn = confusion_matrix(y_test, y_pred_knn)

print("KNN Confusion Matrix:")
print(cm_knn)

```

```

KNN Confusion Matrix:
[[576 159]
 [ 13 287]]

```

```

In [12]: #SVM
svm = SVC(kernel='linear')

svm.fit(X_train_scaled, y_train)

y_pred_svm = svm.predict(X_test_scaled)

print("SVM classification completed!")

```

```

SVM classification completed!

```

```

In [13]: #SVM performance
accuracy_svm = accuracy_score(y_test, y_pred_svm)
precision_svm = precision_score(y_test, y_pred_svm)
recall_svm = recall_score(y_test, y_pred_svm)
f1_svm = f1_score(y_test, y_pred_svm)

print("SVM Performance")
print("-----")
print("Accuracy :", round(accuracy_svm, 4))
print("Precision:", round(precision_svm, 4))
print("Recall   :", round(recall_svm, 4))
print("F1 Score :", round(f1_svm, 4))

```

SVM Performance

Accuracy : 0.9507
Precision: 0.9136
Recall : 0.9167
F1 Score : 0.9151

```
In [14]: #SVM confusion matrix
cm_svm = confusion_matrix(y_test, y_pred_svm)

print("SVM Confusion Matrix:")
print(cm_svm)
```

SVM Confusion Matrix:
[[709 26]
 [25 275]]

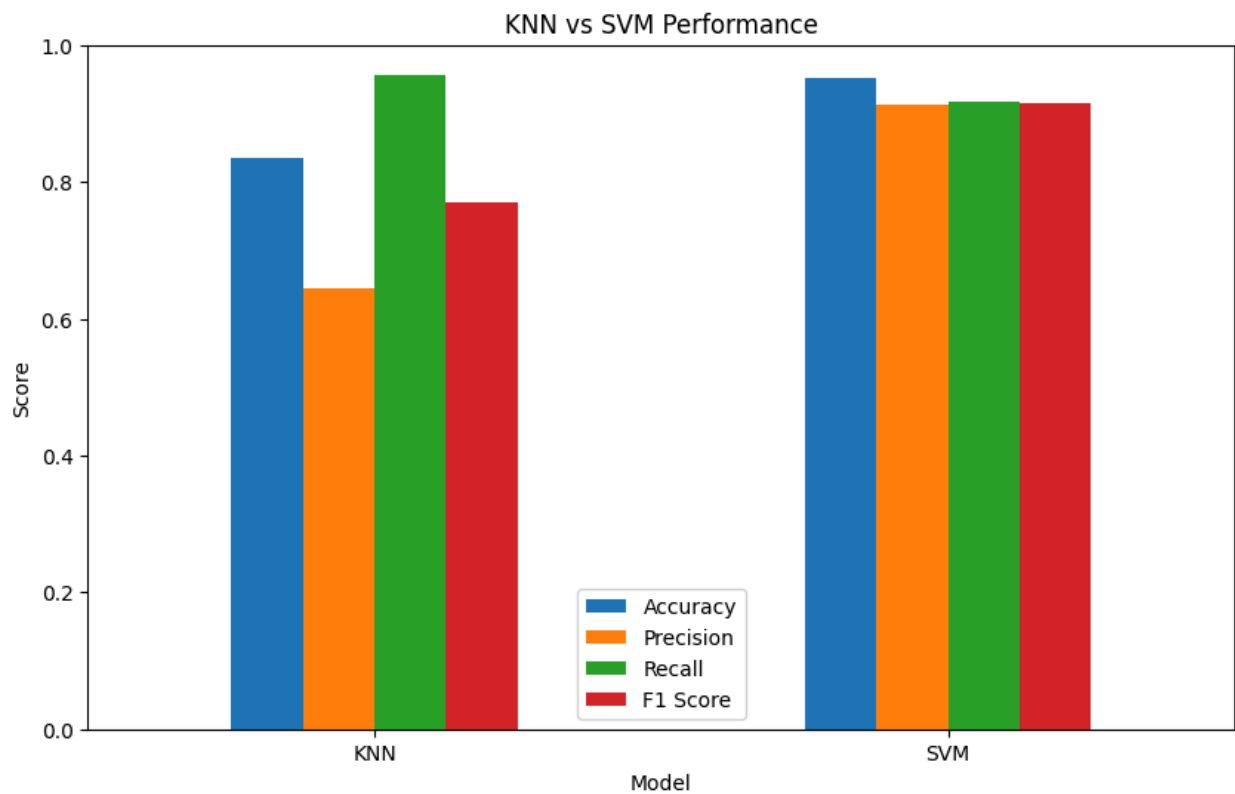
```
In [15]: #Compare KNN and SVM
results = pd.DataFrame({
    'Model': ['KNN', 'SVM'],
    'Accuracy': [accuracy_knn, accuracy_svm],
    'Precision': [precision_knn, precision_svm],
    'Recall': [recall_knn, recall_svm],
    'F1 Score': [f1_knn, f1_svm]
})

print(results)
```

	Model	Accuracy	Precision	Recall	F1 Score
0	KNN	0.833816	0.643498	0.956667	0.769437
1	SVM	0.950725	0.913621	0.916667	0.915141

```
In [16]: #Plot model performance
results.set_index('Model').plot(
    kind='bar',
    figsize=(10, 6)
)

plt.title('KNN vs SVM Performance')
plt.ylabel('Score')
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.legend()
plt.show()
```

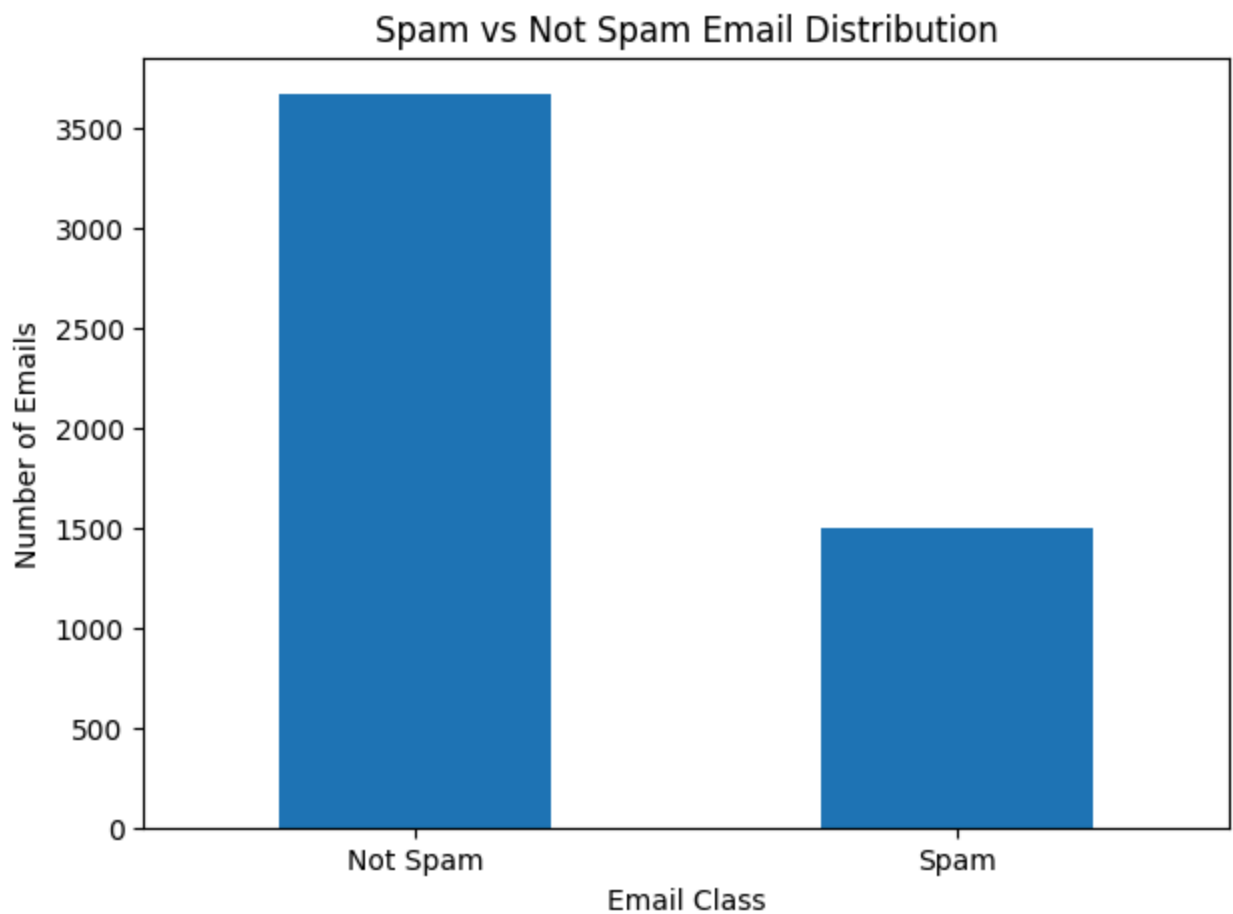


```
In [17]: #Spam vs Not Spam Distribution
plt.figure(figsize=(7, 5))

df.iloc[:, -1].value_counts().plot(kind='bar')

plt.title('Spam vs Not Spam Email Distribution')
plt.xlabel('Email Class')
plt.ylabel('Number of Emails')
plt.xticks([0, 1], ['Not Spam', 'Spam'], rotation=0)

plt.show()
```



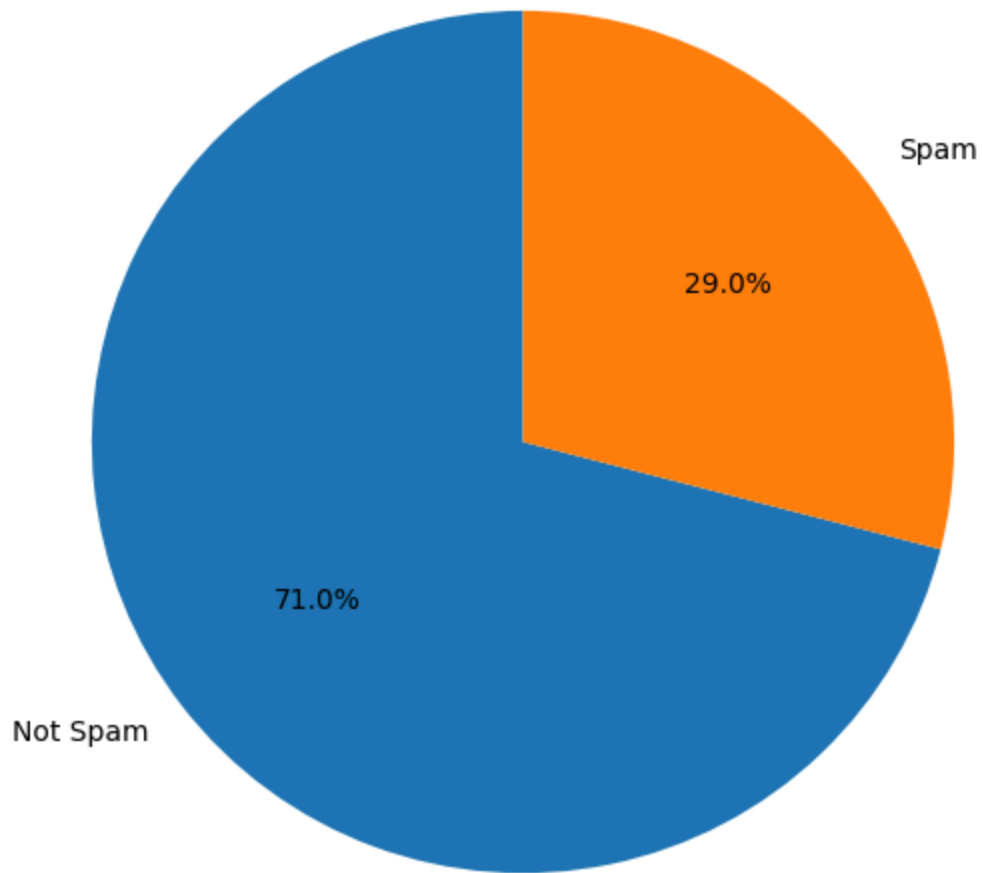
```
In [18]: #Class Distribution Pie Chart
plt.figure(figsize=(7, 7))

df.iloc[:, -1].value_counts().plot(
    kind='pie',
    labels=['Not Spam', 'Spam'],
    autopct='%1.1f%%',
    startangle=90
)

plt.title('Email Spam Distribution')
plt.ylabel('')

plt.show()
```

Email Spam Distribution



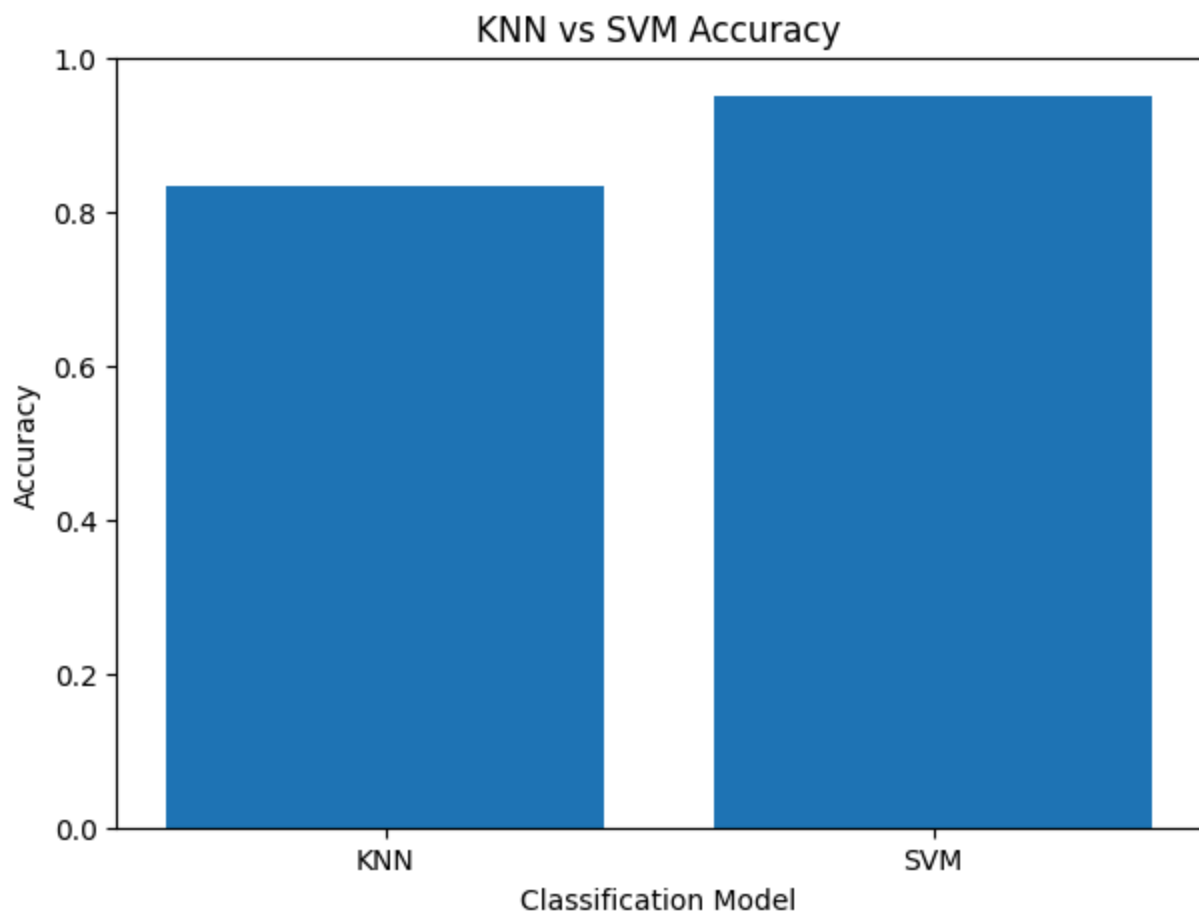
```
In [19]: #KNN vs SVM Accuracy
models = ['KNN', 'SVM']
accuracy = [accuracy_knn, accuracy_svm]

plt.figure(figsize=(7, 5))

plt.bar(models, accuracy)

plt.title('KNN vs SVM Accuracy')
plt.xlabel('Classification Model')
plt.ylabel('Accuracy')
plt.ylim(0, 1)

plt.show()
```



```
In [20]: #KNN vs SVM Precision, Recall and F1
metrics = ['Precision', 'Recall', 'F1 Score']

knn_values = [
    precision_knn,
    recall_knn,
    f1_knn
]

svm_values = [
    precision_svm,
    recall_svm,
    f1_svm
]

x = np.arange(len(metrics))
width = 0.35

plt.figure(figsize=(9, 5))

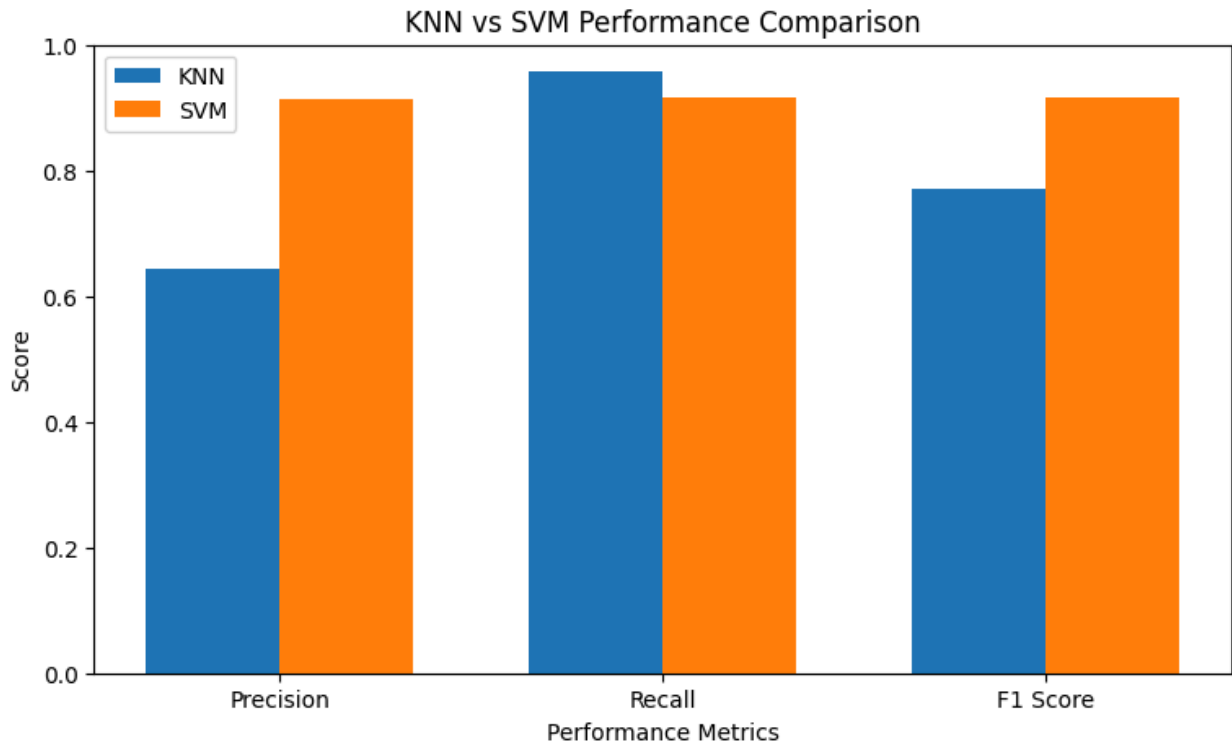
plt.bar(x - width/2, knn_values, width, label='KNN')
plt.bar(x + width/2, svm_values, width, label='SVM')

plt.xlabel('Performance Metrics')
plt.ylabel('Score')
```

```
plt.title('KNN vs SVM Performance Comparison')

plt.xticks(x, metrics)
plt.ylim(0, 1)
plt.legend()

plt.show()
```



```
In [21]: #Final Model Comparison Plot
results = pd.DataFrame({
    'Model': ['KNN', 'SVM'],
    'Accuracy': [accuracy_knn, accuracy_svm],
    'Precision': [precision_knn, precision_svm],
    'Recall': [recall_knn, recall_svm],
    'F1 Score': [f1_knn, f1_svm]
})

results_plot = results.set_index('Model')

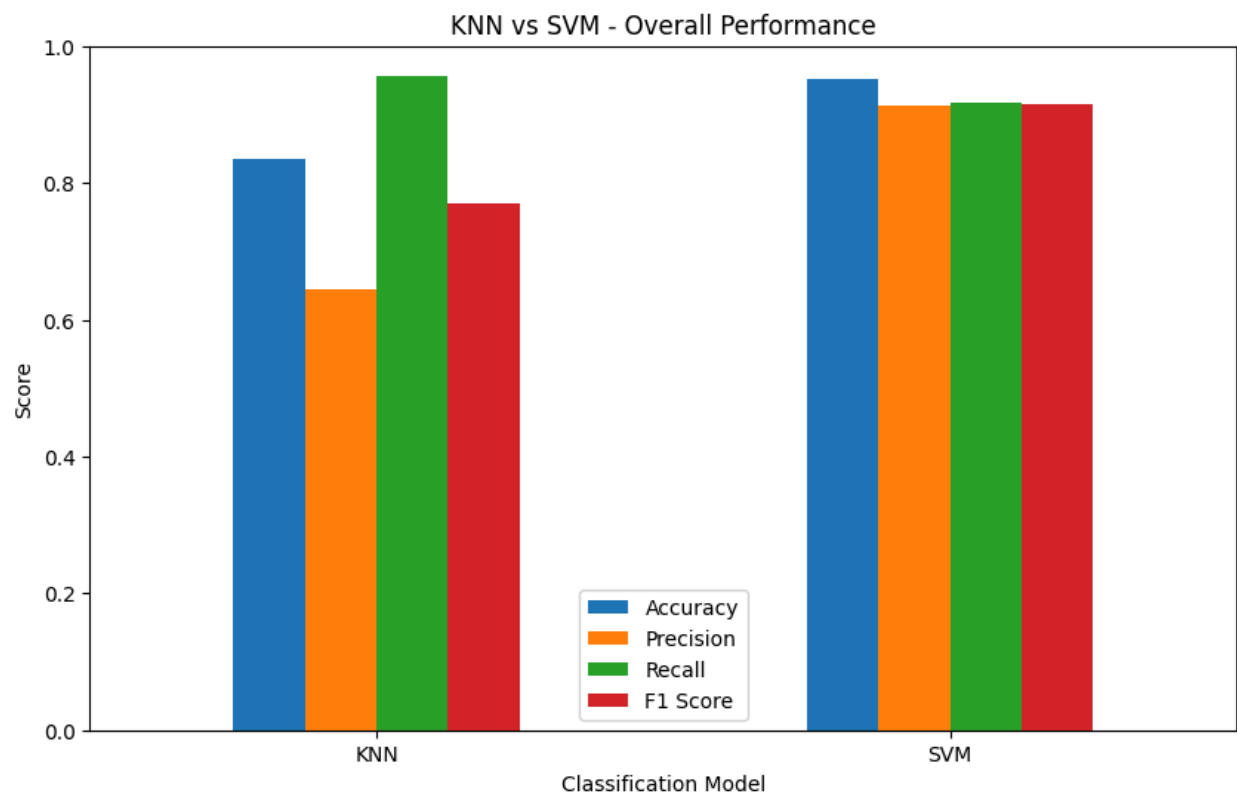
plt.figure(figsize=(10, 6))

results_plot.plot(kind='bar', figsize=(10, 6))

plt.title('KNN vs SVM - Overall Performance')
plt.xlabel('Classification Model')
plt.ylabel('Score')
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.legend()
```

```
plt.show()
```

<Figure size 1000x600 with 0 Axes>



In []: